

Laboratorio de Computacion Grafica 1

Practica 3

Algoritmos de lineas, circunferencias e intersecciones

Profesor	Dr. Hans Harley Ccacyahuilca Bejar
Estudiante	Efrain Vitorino Marin
Codigo	160337
Curso	Laboratorio Computacion Grafica 1
Repositorio	github.com/lovito99/Practica-3-Grafica
Fecha	27 de abril de 2026

1. Objetivo

Implementar en C++ con OpenGL moderno los algoritmos de rasterizacion de lineas y circunferencias indicados en la practica, y utilizarlos para construir una figura compuesta y una escena con cinco circunferencias aleatorias marcando sus puntos de interseccion.

Los objetivos especificos desarrollados en el repositorio fueron los siguientes:

1. Implementar un algoritmo incremental para el trazado de lineas.
2. Implementar los algoritmos de circunferencia por simetria de dos, cuatro y ocho vias.
3. Implementar el algoritmo de circunferencia por punto medio.
4. Componer una escena bidimensional usando las primitivas rasterizadas.
5. Generar cinco circunferencias con parametros aleatorios y resaltar sus intersecciones.

2. Descripcion general

La aplicacion presenta dos escenas interactivas. La primera dibuja una figura compuesta mediante lineas y circunferencias rasterizadas. La segunda genera cinco circunferencias con parametros aleatorios y marca sus puntos de interseccion. Ambas escenas se renderizan como colecciones de puntos coloreados en una ventana OpenGL.

3. Algoritmos implementados

3.1. Algoritmo de lineas

Se implemento el algoritmo de Bresenham para lineas. Este metodo permite rasterizar segmentos utilizando operaciones enteras, por lo que resulta adecuado para construir la estructura base de la figura compuesta, como el contorno del automovil, las ventanas, el suelo, los rayos del sol y las marcas de interseccion.

3.2. Algoritmos de circunferencia

Se implementaron los siguientes algoritmos de circunferencia:

- Simetria de dos vias.
- Simetria de cuatro vias.
- Simetria de ocho vias.
- Punto medio.

Cada algoritmo fue conservado como una funcion independiente para facilitar su comparacion y reutilizacion. En la escena principal se emplean varios de ellos sobre distintos elementos decorativos y estructurales. En la escena aleatoria se asignan de manera alternada a las cinco circunferencias para comprobar que todas las implementaciones produzcan trazos correctos.

4. Parametros clave y logica del codigo

Los parametros principales observables en el codigo son los siguientes:

- Tamano de ventana: 960 por 720 pixeles.
- Proyeccion ortografica en coordenadas de pantalla para facilitar el trazado 2D.
- Rango de centros aleatorios en la escena 2: eje X entre 220 y 740, eje Y entre 200 y 520.
- Rango de radios aleatorios: entre 70 y 130.
- Tamano del punto dibujado en el shader: 4 pixeles.

La logica central del programa puede resumirse de esta manera:

1. Se inicializa GLFW para la ventana, GLEW para las funciones OpenGL y una proyeccion ortografica para dibujar en 2D.
2. Los algoritmos de lineas y circunferencias generan vertices en memoria mediante la funcion de agregado de puntos.
3. La escena 1 construye una figura fija combinando rectas y circunferencias.
4. La escena 2 genera cinco circunferencias aleatorias, alterna los algoritmos de circunferencia y calcula sus intersecciones con geometria analitica.
5. Los vertices se envian a GPU mediante un VBO y se renderizan con color por vertice.

5.Codigo clave de la tarea

Una parte esencial del trabajo consiste en almacenar cada punto generado por los algoritmos dentro de una estructura de vertice con posicion y color. Esto permite reutilizar la misma ruta de render para lineas, circunferencias e intersecciones.

Fuente: `primitives.h` y `primitives.cpp`

```
1 struct Vertex {
2     float x;
3     float y;
4     float r;
5     float g;
6     float b;
7 };
8
9 void agregarPunto(BufferVertices& vertices, int x, int y, Color color) {
10     vertices.push_back({
11         static_cast<float>(x),
12         static_cast<float>(y),
13         color.r, color.g, color.b
14     });
15 }
```

Para el trazado de circunferencias, uno de los bloques clave es la implementacion por simetria de dos vias, que calcula la coordenada Y a partir de la ecuacion del circulo y dibuja los puntos superior e inferior para cada valor de X.

Fuente: primitivas.cpp

```
1 void circuloDosVias(BufferVertices& vertices, int x0, int y0, int radio, Color
  color) {
2     agregarPunto(vertices, x0 + radio, y0, color);
3     agregarPunto(vertices, x0 - radio, y0, color);
4
5     for (int x = -radio + 1; x <= radio - 1; ++x) {
6         const int y = redondeoRaiz(radio, x);
7         agregarPunto(vertices, x0 + x, y0 + y, color);
8         agregarPunto(vertices, x0 + x, y0 - y, color);
9     }
10 }
```

En el caso de las lineas, el algoritmo de Bresenham avanza en la grilla mediante un termino de error y agrega cada punto de manera incremental.

Fuente: primitivas.cpp

```
1 void lineaBresenham(BufferVertices& vertices, int x0, int y0, int x1, int y1,
  Color color) {
2     int xActual = x0;
3     int yActual = y0;
4     const int deltaX = std::abs(x1 - x0);
5     const int deltaY = std::abs(y1 - y0);
6     int error = deltaX - deltaY;
7
8     while (true) {
9         agregarPunto(vertices, xActual, yActual, color);
10        if (xActual == x1 && yActual == y1) {
11            break;
12        }
13        const int dobleError = error * 2;
14        if (dobleError > -deltaY) {
15            error -= deltaY;
16            xActual += (x0 < x1) ? 1 : -1;
17        }
18        if (dobleError < deltaX) {
19            error += deltaX;
20            yActual += (y0 < y1) ? 1 : -1;
21        }
22    }
23 }
```

Para completar lo solicitado por la practica, tambien se incluyeron las otras tres variantes de circunferencia. La version de cuatro vias aprovecha la simetria respecto a ambos ejes:

Fuente: primitivas.cpp

```
1 void circuloCuatroVias(BufferVertices& vertices, int x0, int y0, int radio,
  Color color) {
```

```

2   agregarPunto(vertices, x0, y0 + radio, color);
3   agregarPunto(vertices, x0, y0 - radio, color);
4   agregarPunto(vertices, x0 + radio, y0, color);
5   agregarPunto(vertices, x0 - radio, y0, color);
6
7   for (int x = 1; x <= radio - 1; ++x) {
8       const int y = redondeoRaiz(radio, x);
9       agregarPunto(vertices, x0 + x, y0 + y, color);
10      agregarPunto(vertices, x0 + x, y0 - y, color);
11      agregarPunto(vertices, x0 - x, y0 + y, color);
12      agregarPunto(vertices, x0 - x, y0 - y, color);
13  }
14 }

```

La version de ocho vias extiende la simetria para dibujar los ocho puntos equivalentes del mismo octante:

Fuente: primitivas.cpp

```

1 void circuloOchoVias(BufferVertices& vertices, int x0, int y0, int radio, Color
   color) {
2     agregarPunto(vertices, x0, y0 + radio, color);
3     agregarPunto(vertices, x0, y0 - radio, color);
4     agregarPunto(vertices, x0 + radio, y0, color);
5     agregarPunto(vertices, x0 - radio, y0, color);
6
7     int x = 1;
8     int y = redondeoRaiz(radio, x);
9
10    while (x < y) {
11        agregarPunto(vertices, x0 + x, y0 + y, color);
12        agregarPunto(vertices, x0 + x, y0 - y, color);
13        agregarPunto(vertices, x0 - x, y0 + y, color);
14        agregarPunto(vertices, x0 - x, y0 - y, color);
15        agregarPunto(vertices, x0 + y, y0 + x, color);
16        agregarPunto(vertices, x0 + y, y0 - x, color);
17        agregarPunto(vertices, x0 - y, y0 + x, color);
18        agregarPunto(vertices, x0 - y, y0 - x, color);
19
20        ++x;
21        y = redondeoRaiz(radio, x);
22    }
23 }

```

El algoritmo de punto medio utiliza una variable de decision incremental para evitar recalcular la ecuacion del circulo en cada iteracion:

Fuente: primitivas.cpp

```

1 void circuloPuntoMedio(BufferVertices& vertices, int x0, int y0, int radio,
   Color color) {
2     double hm = 1.25 - static_cast<double>(radio);
3     int x = 0;
4     int y = -radio;
5

```

```

6   agregarPunto(vertices, x0, y0 + radio, color);
7   agregarPunto(vertices, x0, y0 - radio, color);
8   agregarPunto(vertices, x0 + radio, y0, color);
9   agregarPunto(vertices, x0 - radio, y0, color);
10
11  while (x < -(y + 1)) {
12      if (hm < 0.0) {
13          hm += (2.0 * static_cast<double>(x)) + 3.0;
14      } else {
15          hm += (2.0 * static_cast<double>(x)) + (2.0 * static_cast<double>(y))
16              + 5.0;
17          ++y;
18      }
19      ++x;
20      dibujarOchoPuntos(vertices, x0, y0, x, y, color);
21  }

```

La segunda parte de la tarea pide cinco circunferencias aleatorias y sus intersecciones. Para ello se calculan geométricamente los puntos de corte entre pares de circunferencias:

Fuente: primitivas.cpp

```

1  std::vector<PuntoReal> calcularIntersecciones(const Circulo& primero, const
    Circulo& segundo) {
2      const double dx = static_cast<double>(segundo.centroX - primero.centroX);
3      const double dy = static_cast<double>(segundo.centroY - primero.centroY);
4      const double distancia = std::hypot(dx, dy);
5
6      if (distancia > static_cast<double>(primero.radio + segundo.radio)) {
7          return {};
8      }
9
10     const double radioA = static_cast<double>(primero.radio);
11     const double radioB = static_cast<double>(segundo.radio);
12     const double a = ((radioA * radioA) - (radioB * radioB) + (distancia *
        distancia)) / (2.0 * distancia);
13     const double h = std::sqrt(std::max(0.0, (radioA * radioA) - (a * a)));
14
15     const double baseX = static_cast<double>(primero.centroX) + (a * dx /
        distancia);
16     const double baseY = static_cast<double>(primero.centroY) + (a * dy /
        distancia);
17
18     std::vector<PuntoReal> puntos;
19     puntos.push_back({
20         static_cast<float>(baseX - dy * h / distancia),
21         static_cast<float>(baseY + dx * h / distancia)
22     });
23     if (h > 0.0) {
24         puntos.push_back({
25             static_cast<float>(baseX + dy * h / distancia),
26             static_cast<float>(baseY - dx * h / distancia)

```

```

27     });
28 }
29 return puntos;
30 }

```

La escena aleatoria se arma alternando los algoritmos de circunferencia y dibujando un marcador sobre cada interseccion encontrada:

Fuente: `escenas.cpp`

```

1 BufferVertices crearEscenaIntersecciones() {
2     BufferVertices vertices;
3     const std::vector<Circulo> circulos = generarCirculos(semilla);
4     const std::array<AlgoritmoCirculo, 4> algoritmos = {
5         circuloDosVias,
6         circuloCuatroVias,
7         circuloOchoVias,
8         circuloPuntoMedio
9     };
10
11     for (std::size_t indice = 0; indice < circulos.size(); ++indice) {
12         const Circulo& circulo = circulos[indice];
13         const AlgoritmoCirculo algoritmo = algoritmos[indice % algoritmos.size()];
14         algoritmo(vertices, circulo.centroX, circulo.centroY, circulo.radio,
15             circulo.color);
16     }
17
18     for (std::size_t i = 0; i < circulos.size(); ++i) {
19         for (std::size_t j = i + 1; j < circulos.size(); ++j) {
20             const auto puntos = calcularIntersecciones(circulos[i], circulos[j]);
21             for (const PuntoReal& punto : puntos) {
22                 dibujarMarcador(vertices, static_cast<int>(std::lround(punto[0])),
23                     static_cast<int>(std::lround(punto[1])), colorRojo);
24             }
25         }
26     }
27     return vertices;
28 }

```

6. Flujo de vertices de CPU a GPU

El recorrido de los vertices desde CPU hasta GPU en la aplicacion sigue una secuencia clara:

1. En CPU, los algoritmos generan puntos y los almacenan en un contenedor de tipo `std::vector<Vertice>`.
2. Cada `Vertice` contiene posicion 2D y color RGB.
3. Cuando una escena esta lista, sus vertices se copian al buffer de OpenGL mediante `glBufferData`.

4. El VAO define como interpretar esos datos: la posicion entra por `location = 0` y el color por `location = 1`.
5. El vertex shader transforma la posicion con la matriz de proyeccion y reenvia el color al fragment shader.
6. Finalmente, OpenGL rasteriza los puntos con `glDrawArrays(GL_POINTS, ...)`.

El envio del arreglo de vertices desde CPU a GPU se realiza en el siguiente bloque:

Fuente: main.cpp

```
1 void cargarEscena(GLuint vbo, const BufferVertices& vertices) {
2     glBindBuffer(GL_ARRAY_BUFFER, vbo);
3     glBufferData(
4         GL_ARRAY_BUFFER,
5         static_cast<GLsizeiptr>(vertices.size() * sizeof(Vertex)),
6         vertices.data(),
7         GL_STATIC_DRAW);
8 }
```

La configuracion de atributos del vertice se hace en main.cpp mediante el VAO y el VBO:

Fuente: main.cpp

```
1 glBindVertexArray(vao);
2 glBindBuffer(GL_ARRAY_BUFFER, vbo);
3 glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex),
4     reinterpret_cast<void*>(0));
5 glEnableVertexAttribArray(0);
6 glVertexAttribPointer(
7     1, 3, GL_FLOAT, GL_FALSE,
8     sizeof(Vertex), reinterpret_cast<void*>(2 * sizeof(float)));
9 glEnableVertexAttribArray(1);
```

En GPU, el shader de vertices recibe posicion y color y prepara la salida para rasterizacion:

Fuente: shader.vert

```
1 layout (location = 0) in vec2 aPos;
2 layout (location = 1) in vec3 aColor;
3 uniform mat4 projection;
4
5 out vec3 vertexColor;
6
7 void main() {
8     gl_Position = projection * vec4(aPos, 0.0, 1.0);
9     gl_PointSize = 4.0;
10    vertexColor = aColor;
11 }
```


7. Estructura del repositorio

Archivo	Funcion dentro del proyecto
main.cpp	Inicializacion de GLFW, GLEW y OpenGL, configuracion de buffers y cambio entre escenas
primitivas.h / primitivas.cpp	Definicion de estructuras y algoritmos de lineas, circunferencias e intersecciones
escenas.h / escenas.cpp	Construccion de la escena compuesta y de la escena aleatoria
Shaders .vert y .frag	Shaders para dibujar puntos con color por vertice
README.md	Instrucciones generales de compilacion y uso
informe/informe.tex	Documento tecnico de la practica

8. Desarrollo de la solucion

La figura compuesta utiliza lineas para carretera, carroceria, puertas, ventanas, rayos y detalles, mientras que las circunferencias se usan para ruedas, sol, arbol y elementos auxiliares. La escena de intersecciones genera automaticamente cinco circunferencias con una semilla distinta en cada ejecucion y comprueba que existan cruces visibles antes de aceptar el conjunto.

Las intersecciones entre dos circunferencias se obtienen a partir de la distancia entre centros y de la solucion geometrica del sistema. Cuando existe uno o dos puntos de corte, estos se remarcan con cruces dibujadas por el algoritmo de Bresenham.

En la escena aleatoria, cada circunferencia se dibuja usando uno de los algoritmos disponibles de forma alternada. Asi, durante una sola ejecucion se visualiza el resultado conjunto de:

- Simetria de dos vias.
- Simetria de cuatro vias.
- Simetria de ocho vias.
- Punto medio.

9. Imagen generada

La Figura 1 muestra la salida obtenida para la escena principal del programa.

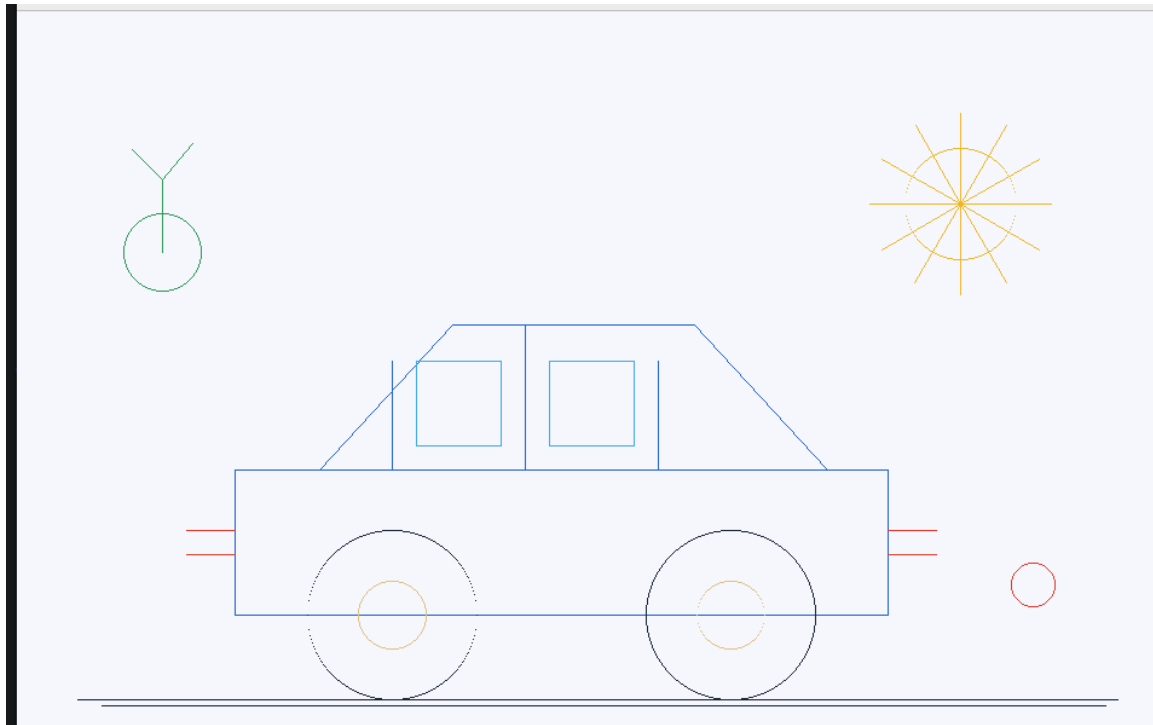


Figura 1: Escena compuesta generada por la aplicacion OpenGL.

10. Compilacion y ejecucion

Desde la raiz del proyecto se utilizan los siguientes comandos:

```
cmake -S . -B build
cmake --build build
./build/circunferencias
```

Durante la ejecucion del programa se dispone de los siguientes controles:

- Tecla 1: muestra la figura compuesta.
- Tecla 2: muestra cinco circunferencias aleatorias con intersecciones.
- Tecla R: regenera la escena aleatoria para obtener una nueva configuracion.

11. Resultados obtenidos

Los resultados obtenidos fueron los siguientes:

- La aplicacion compilo correctamente en Linux usando CMake.
- La escena compuesta mostro una imagen consistente formada por lineas y circunferencias rasterizadas.
- La escena aleatoria confirmo el funcionamiento del calculo de intersecciones entre circunferencias.

- Los cuatro algoritmos de circunferencia produjeron trazos validos dentro de la misma aplicacion.

El repositorio de trabajo utilizado para el desarrollo y seguimiento del proyecto es el siguiente:

<https://github.com/lovito99/Practica-3-Grafica>

12. Conclusiones

La practica permitio implementar y comparar algoritmos clasicos de graficacion por computadora en un entorno OpenGL moderno. Se verifico que los enfoques basados en simetria y el algoritmo de punto medio permiten generar circunferencias correctas, mientras que Bresenham proporciona una base eficiente para el trazado de lineas.

Ademas, el trabajo mostro que estas primitivas pueden reutilizarse para construir escenas completas y para resolver tareas geometricas adicionales, como la deteccion y visualizacion de intersecciones entre circunferencias. El resultado final integra implementacion, visualizacion interactiva y documentacion tecnica dentro del mismo repositorio.